

RootsTalk

Localisation Strategy

End-to-end language handling — April 2026

This document defines the complete localisation architecture for RootsTalk. It covers how language is selected, stored, transmitted, and rendered across all surfaces; the four categories of translatable content and how each is handled; the translation workflow for expert-authored content; RTL language support; number and date formatting; and the technical implementation requirements for both frontend and backend. This document must be read by any developer working on language-related features.

1. Supported Languages

RootsTalk supports thirteen languages. English is the platform default and is always active. The twelve Indian languages are enabled by the SA from the Admin Portal as translations become ready.

Code	Language	Script Family	Direction	Status	Special Handling
en	English	Latin	LTR	Always ACTIVE — cannot disable	Fallback for all untranslated content
hi	Hindi	Devanagari	LTR	SA-toggled	Shared script with Marathi
ta	Tamil	Tamil	LTR	SA-toggled	—
te	Telugu	Telugu	LTR	SA-toggled	—
kn	Kannada	Kannada	LTR	SA-toggled	—
ml	Malayalam	Malayalam	LTR	SA-toggled	Long script — test UI overflow
mr	Marathi	Devanagari	LTR	SA-toggled	Shared script with Hindi
gu	Gujarati	Gujarati	LTR	SA-toggled	—
pa	Punjabi	Gurmukhi	LTR	SA-toggled	—
or	Odia	Odia	LTR	SA-toggled	—
bn	Bengali	Bengali	LTR	SA-toggled	Shared script with Assamese
as	Assamese	Bengali	LTR	SA-toggled	Shared script with Bengali
ur	Urdu	Perso-Arabic	RTL	SA-toggled	Only RTL language. Requires full RTL layout.

■ **Rule: Urdu is the only RTL (right-to-left) language in the set. The frontend must switch the entire layout direction when Urdu is selected. See Section 6.**

■ **Note: Languages are shown to farmers in their own script in the language selector dropdown — e.g. Kannada shown as 'ಕನ್ನಡ', Tamil as 'தமிழ்'. The farmer must be able to recognise and tap their language before anything is rendered in that language.**

2. Language Scope by Surface

Surface	Localised?	Languages	Rationale
Farmer PWA	Fully localised	All SA-enabled languages	Primary farmer-facing surface. Farmers in rural India may not read English.
Dealer PWA	Fully localised	All SA-enabled languages	Dealers operate in local languages. Brand names and practice names must render correctly.
Facilitator PWA	Fully localised	All SA-enabled languages	Same as Dealer.
FarmPundit PWA	Partially localised	All SA-enabled languages for farmer-submitted content; UI in English and expert's profile language	FarmPundits are educated professionals. Query content from farmers shown in farmer's language. FarmPundit response interface in English.
Admin Portal (Neytiri)	English only	English	B2B internal tool. All Neytiri staff operate in English.

Surface	Localised?	Languages	Rationale
Client Portal	English only	English	B2B tool for company CA and Subject Experts. Email-based auth. English throughout.
Public crop history QR page	English only (Phase 1)	English	Scanned by buyers, auditors, inspectors. English is the default. Multi-language planned for future phase.
Emails	English only	English	Operational and administrative. English is the standard.
SMS / OTP	English only	English	Short alphanumeric OTP. No translation needed.

3. Content Categories and Translation Approach

Not all content in RootsTalk is translated the same way. There are four distinct content categories, each with its own translation source and workflow.

Category	Examples	Translation Source	Review Required?	Fallback
Cosh-sourced reference data	Crop names, district names, practice names, application methods, symptom names, brand names, problem group names	Cosh maintains translations in all supported languages. Delivered via the Cosh sync API already translated.	No — translations come pre-verified from Cosh	English from Cosh
System UI strings	Button labels, navigation items, status badges, empty state messages, error messages, confirmation dialogs, notification text	Neytiri content team provides translations. Stored in i18n JSON files in the frontend codebase.	Yes — Neytiri team reviews all i18n strings before enabling a language	English key value
Expert-authored free-text advisory content	Practice instructions, General Instruction titles and bodies, Package descriptions, PoP selection confirmation text	Auto-translated by Python microservice (IndicTrans2 or Google Translate) on save. Subject Expert reviews and can edit or retranslate.	Yes — Expert must mark as Approved per language before it is served to farmers	Auto-translation shown to farmer with a 'Translation pending review' indicator (subtle, non-alarming)
Custom Parameters and Variables	Expert-created guided elimination questions and answer options not available in Cosh	Auto-translated on save. Creator (Subject Expert or CM) reviews and approves per language.	Yes — Creator approves per language. Reset to Pending if text is edited.	Auto-translation shown with 'Translation pending review' indicator

4. Language Storage and User Preference

4.1 Where Language Preference Is Stored

- The user's chosen language is stored in `users.language_code` (VARCHAR, BCP-47 code, e.g. 'kn', 'ta', 'en').
- Language selection is the first action in PWA registration. It cannot be skipped.

- The farmer can change their language at any time from the header dropdown. The change is saved immediately to `users.language_code` and the PWA re-renders.
- The language preference persists across sessions. When the farmer logs in on a new device, the PWA loads in their saved language.

4.2 How Language Is Communicated to the Backend

Every authenticated API request from the PWA includes the user's language in the request header:

```
Accept-Language: kn // ISO 639-1 language code from users.language_code
```

- The backend reads this header on every request and serves translated content for that language.
- If the user's language is not in the header (legacy client, unauthenticated request), the backend defaults to English.
- The backend does NOT re-query the database for the user's `language_code` on every request — the header is authoritative. This avoids an extra DB lookup per request.

4.3 Language Selector — UX Rules

Language Selector — Confirmed Design Rules

- Placed in the PWA header. Visible from every screen. Never buried in Profile or Settings.
- Shows the current language abbreviation or a globe icon.
- Tapping opens a compact overlay list of all SA-enabled languages.
- Each language displayed in its own script: '■■■■■■' not 'Tamil', '■■■■■■' not 'Kannada'.
- English always shown as 'English' (its own name in Latin script).
- Selecting a language: immediate re-render of entire PWA. No page reload required.
- New SA-enabled languages appear silently in this dropdown. No push notification to existing users.
- Placement rationale: if the farmer selected the wrong language accidentally, they cannot navigate to fix it — because everything is in the wrong language. Header placement makes the selector always findable regardless of what language the UI is in.

5. Translation Workflows

5.1 Cosh-Sourced Data — No Action Required

All data that originates in Cosh (crop names, brand names, location names, symptom names, application methods, etc.) is already translated by the Cosh team and delivered via the sync API. RootsTalk stores the translated values in the `cosh_reference_cache` table alongside the Cosh ID. No translation action is required from RootsTalk Subject Experts or developers for this category.

■ *Note: The `cosh_reference_cache` table stores one row per language per Cosh entity. The backend looks up the correct language row based on the `Accept-Language` header.*

5.2 System UI Strings — i18n Files

All fixed UI text (buttons, labels, messages, empty states, confirmations) is managed as JSON key-value pairs in the frontend codebase. One file per language.

```
// File structure /translations/ en.json // English (source of truth for all keys) kn.json //
Kannada ta.json // Tamil hi.json // Hindi ... // One file per enabled language
```

i18n String Management Rules

- English (en.json) is the source of truth. Every key must exist in en.json before being added to other language files.
- When a new UI string is added during development: add it to en.json first with the English value. Add the same key to all other language files immediately, using the English value as a placeholder. Mark with a TODO comment: // TODO: translate
- Neytiri content team provides translations for each language. Translations are committed to the language files. No database involved.
- When a new language is enabled by the SA: Neytiri must ensure all keys in that language file are translated (not placeholder English). The language must not be enabled until i18n strings are ready.
- The frontend i18n library (next-intl or react-intl) falls back to the English value if a key is missing in the target language file.
- Never hard-code display strings in React components. Always use the i18n translation function: t('key_name').

```
// Correct usage in React component import { useTranslations } from 'next-intl'; function
AdvisoryScreen() { const t = useTranslations('advisory'); return {t('order_now')}; // Do NOT:
return Order Now; }
```

5.3 Expert-Authoring Advisory Content — Auto-Translation Workflow

When a Subject Expert writes free-text content (practice instructions, package descriptions, guidance text), this text must be made available in all SA-enabled languages. The workflow is:

Step	Actor	Action
1	Subject Expert	Writes free-text content in English (or their preferred language) in the Client Portal. Saves the content.
2	System (auto)	Detects that this is free-text content. Calls the Python translation microservice. The microservice calls IndicTrans2 (for Indic languages) or Google Translate (for English ↔ Indic). Stores translations in the element_translations table (one row per language).
3	System	Marks all translations as status='PENDING'. Notifies the Subject Expert: 'Translations generated. Please review.'
4	Subject Expert	Opens the translation review panel in the Client Portal. Sees the auto-translated text for each enabled language. For each language: can Approve as-is, or Edit and then Approve.
5	System	Marks approved languages as status='APPROVED'. Serves APPROVED translations to farmers. For PENDING languages: serves the auto-translation with a subtle 'translation pending' indicator.
6	Subject Expert (ongoing)	Can return to any content at any time and update translations. Editing the original English text resets all translations to PENDING and re-triggers auto-translation.

■ *Note: The 'translation pending' indicator shown to farmers must be subtle and non-alarming. A small italic note below the text: 'Translation in progress' is sufficient. The farmer still sees the content — they are not blocked.*

■ **Rule: A PoP can be published even if not all translations are APPROVED. Publishing is not gated on translation completion. However, farmers in languages where translations are PENDING will see auto-translated content with the pending indicator.**

5.4 Custom Parameters and Variables — Creator-Approved Translation

Custom Parameters and Variables (Design Addendum §A1) follow the same auto-translate workflow as expert-authored content, with one additional rule:

Custom Parameter/Variable Translation — Additional Rules

• Auto-translation is triggered immediately on save of any new Custom Parameter or Variable.
• The creator (Subject Expert or CM) is shown all auto-translations for review immediately after saving.
• Each language has its own Approved/Pending status. Creator approves per language.
• Editing the Variable text resets ALL language translations to Pending and re-triggers auto-translation for all languages simultaneously.
• Custom Parameters and Variables are used in the guided elimination flow shown to farmers — these must be clear and accurate. Creators must prioritise approving translations for languages where farmers in their region are likely to subscribe.
• Until a language is Approved: the auto-translation is shown to farmers in that language. Approved translations always take precedence.

5.5 When a New Language Is Enabled by the SA

When the SA enables a new language, this triggers a batch translation event for all existing expert-authored content:

What Happens	Timeline	Notes
System identifies all existing free-text content that does not have a translation for the new language code	Immediately on SA enabling the language	Queries element_translations, parameter_translations, variable_translations for missing language rows
Python microservice called in batch for all missing translations	Within minutes (batch job)	Large systems may queue this — handle gracefully with a progress indicator in Admin Portal
New translations stored as PENDING	On completion of batch	Subject Experts see them as Pending Review in the Client Portal
New language appears in PWA language selector for all users	Immediately on SA enabling	No wait for translations. Pending content shows auto-translation with indicator.
Subject Experts notified: new language available for review	After batch completes	Notification in Client Portal: 'X translations in [Language] need your review.'

6. RTL Layout — Urdu

Urdu is the only right-to-left language in the RootsTalk supported set. When a user selects Urdu, the entire PWA layout must mirror: text alignment, flex direction, icons, navigation, and margin/padding conventions all reverse.

6.1 What Changes in RTL Mode

Element	LTR (Default)	RTL (Urdu)
Text alignment	Left-aligned	Right-aligned
Flex direction	Row (left to right)	Row-reverse (right to left)
Padding/Margin	Left side for indent	Right side for indent
Back arrow navigation	Points left (←)	Points right (→)
Swipe gestures	Swipe right = back	Swipe left = back
Number display	Left to right	Left to right — numbers stay LTR even in RTL context (standard for Urdu)
Icon placement	Left of label	Right of label
Progress bars	Fill left to right	Fill right to left

Element	LTR (Default)	RTL (Urdu)
Scrollbar	Right edge	Left edge

6.2 Implementation Approach

- Set the HTML dir attribute: when language is 'ur', set . This triggers automatic CSS RTL mirroring for most layout properties.
- Use CSS logical properties throughout: margin-inline-start instead of margin-left, padding-inline-end instead of padding-right. These flip automatically with dir=rtl.
- Use Tailwind CSS RTL plugin or Next.js built-in RTL support. Do NOT write separate RTL stylesheets.
- Numbers in Urdu: Eastern Arabic numerals (٠١٢٣) are used in Urdu script, but Indian users reading Urdu commonly use Hindu-Arabic (0–9). Use standard Hindu-Arabic numerals for prices, quantities, and dates. Test with Urdu-reading users before launch.
- Font: Urdu requires a Nastaliq or Naskh font. Google Font: Noto Nastaliq Urdu. Load conditionally only when language='ur'.

```
// Conditional font loading in Next.js if (language === 'ur') {
document.documentElement.setAttribute('dir', 'rtl'); // Load Noto Nastaliq Urdu font } else {
document.documentElement.setAttribute('dir', 'ltr'); }
```

■ *Note: Urdu RTL testing must be done on a real device with an Urdu keyboard installed. Browser RTL simulation is not sufficient for catching layout issues. Schedule a dedicated Urdu UI review before enabling Urdu in production.*

7. Numbers, Dates, and Currency Formatting

7.1 Number Formatting — Indian Numbering System

India uses a distinct numbering system: groups of two digits after the first three (1,00,000 = one lakh, not 100,000). RootsTalk must use Indian number formatting for all displayed quantities.

Value	International Format	Indian Format (Correct)	Context
100000	1,00,000	1,00,000	Subscription pool units, farm area in large farms
500000	500,000	5,00,000	Larger quantities
1000000	1,000,000	10,00,000	Ten lakhs
199	199	199	No grouping needed below 1000
2.5	2.5	2.5	Decimal separator remains a period

```
// Use Intl.NumberFormat with Indian locale const fmt = new Intl.NumberFormat('en-IN');
fmt.format(500000); // Returns '5,00,000'
```

7.2 Currency — Indian Rupee

- All prices shown with the ₹ symbol (Unicode U+20B9). Not 'Rs.' or 'INR'.
- Subscription fee: ₹199 (no decimal places for whole amounts).
- Expert query fee: ₹25.
- Input prices: ₹[amount] with 2 decimal places if not a whole number.
- Price format: ₹2,500.00 (currency symbol before number, Indian grouping, 2 decimal places for prices).

7.3 Date Formatting

Indian users read dates in DD-MM-YYYY format. All dates displayed in the PWA must follow this convention.

Context	Format	Example
Date displayed to farmer (advisory dates, order dates)	DD MMM YYYY	15 Jun 2026 (month abbreviated in farmer's language)
Date in dealer/facilitator packing list	DD-MM-YYYY	15-06-2026
Manufacture/Expiry date on QR code page	DD-MM-YYYY	01-03-2026
Date in emails and Admin Portal	DD MMM YYYY	15 Jun 2026
API date format (internal)	YYYY-MM-DD (ISO 8601)	2026-06-15 (stored in UTC, displayed in IST)

■ *Note: All dates stored in the database as TIMESTAMPTZ in UTC. The backend converts to IST (UTC+5:30) for display. The frontend does not perform timezone conversion — it displays the date string returned by the API.*

7.4 Month Names in Indian Languages

When displaying month names in farmer's language (e.g. 'Jun' in advisory dates), the i18n files must include abbreviated and full month names in each supported language. Example:

```
// en.json { "months": { "short": ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"] } } // kn.json (Kannada) { "months": { "short": ["ಜನ", "ಫೆಬ್ರ", "ಮಾರ್ಚ್", "ಏಪ್ರಿಲ್", "ಮೇ", "ಜೂನ್", "ಜುಲೈ", "ಆಗಸ್ಟ್", "ಸೆಪ್ಟೆ", "ಅಕ್ಟೋ", "ನೋವೆ", "ಡಿಸೆಂ"] } }
```

8. Backend Implementation

8.1 Translation Tables

Free-text advisory content translations are stored in a set of `_translations` tables mirroring the source tables:

Translation Table	Source Table	Stores
<code>element_translations</code>	<code>elements</code>	Translated value for free-text elements (instructions, titles). One row per element per language.
<code>parameter_translations</code>	<code>parameters</code>	Translated name for CUSTOM parameters. One row per parameter per language.
<code>variable_translations</code>	<code>variables</code>	Translated name for CUSTOM variables. One row per variable per language.
<code>conditional_question_translations</code>	<code>conditional_questions</code>	Translated question text. One row per question per language.

Schema pattern for every `_translations` table:

```
CREATE TABLE element_translations ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), element_id UUID NOT NULL REFERENCES elements(id), language_code VARCHAR(10) NOT NULL, -- BCP-47 code translated_value TEXT NOT NULL, -- Auto-translated or expert-edited text translation_status VARCHAR(20) NOT NULL -- 'PENDING' | 'APPROVED' DEFAULT 'PENDING', approved_by UUID REFERENCES users(id), -- NULL until approved_at TIMESTAMPTZ, updated_at TIMESTAMPTZ DEFAULT now(), UNIQUE (element_id, language_code) );
```

8.2 API Response Language Handling

When the backend assembles an API response containing translatable content, it uses the `Accept-Language` header to select the correct translation:

```
# FastAPI service pattern def get_element_value(element_id: str, lang: str) -> str: # 1. Try
APPROVED translation for requested language approved = db.query( ElementTranslation ).filter(
ElementTranslation.element_id == element_id, ElementTranslation.language_code == lang,
ElementTranslation.translation_status == 'APPROVED' ).first() if approved: return
approved.translated_value # 2. Try PENDING auto-translation (serve with indicator) pending =
db.query(ElementTranslation).filter( ElementTranslation.element_id == element_id,
ElementTranslation.language_code == lang ).first() if pending: return pending.translated_value #
Frontend adds 'pending' indicator # 3. Fall back to English english =
db.query(ElementTranslation).filter( ElementTranslation.element_id == element_id,
ElementTranslation.language_code == 'en' ).first() return english.translated_value if english else
elements.value
```

8.3 The Python Translation Microservice

A dedicated Python microservice handles all auto-translation calls. It is separate from the main FastAPI application and runs as a Celery worker task.

Translation Provider	Used For	Why
IndicTrans2 (AI4Bharat)	English → all Indic languages	Open-source model specifically trained on Indian languages. Higher quality for agricultural terminology in Indian languages than general-purpose MT models.
Google Translate API	Fallback if IndicTrans2 is unavailable; languages not well-covered by IndicTrans2	Reliable fallback. Broad language coverage.
Both providers evaluated	Final decision during Phase 3 development	Run quality comparison on RootsTalk-specific agricultural content before committing to one provider.

```
# Translation worker (Celery task) @celery.task def translate_element(element_id: str,
source_text: str, source_lang: str, target_langs: list[str]): for lang in target_langs: if lang ==
source_lang: continue # No self-translation translated = call_indictrans2(source_text,
source_lang, lang) # Fallback to Google Translate if IndicTrans2 fails if not translated:
translated = call_google_translate(source_text, source_lang, lang) if translated:
upsert_translation( element_id=element_id, language_code=lang, value=translated, status='PENDING'
)
```

■ *Note: Translation calls are always asynchronous (Celery tasks). Never block the API response waiting for translation. The save completes immediately; translations appear in the review panel within minutes.*

8.4 Cosh Reference Cache — Language Lookup

Cosh reference data (crop names, application methods, etc.) is stored in the `cosh_reference_cache` table with one row per language per entity. The backend looks up the language-specific value using a straightforward join:

```
# Getting a crop name in the user's language def get_crop_name(crop_cosh_id: str, lang: str) ->
str: result = db.execute( "SELECT display_value FROM cosh_reference_cache " "WHERE cosh_id = :id
AND language_code = :lang", {"id": crop_cosh_id, "lang": lang} ).first() if result: return
result.display_value # Fallback to English result_en = db.execute( "SELECT display_value FROM
cosh_reference_cache " "WHERE cosh_id = :id AND language_code = 'en'", {"id": crop_cosh_id}
).first() return result_en.display_value if result_en else crop_cosh_id
```

9. Frontend Implementation

9.1 i18n Library

- Use next-intl (for Next.js App Router) as the i18n library. It supports server-side rendering, has minimal client-side JavaScript footprint, and integrates cleanly with Next.js route structure.

- Alternative: react-intl. Either is acceptable. Decide during Phase 4.
- Do not use browser-native Intl API alone — it does not handle the key-based string lookup that system UI strings require.

9.2 Language Detection and Initialisation

```
// Language initialisation order // 1. Check JWT / session: user's saved language_code from
database // 2. If no session (first-time user): show language selection screen // 3. Never infer
from browser Accept-Language header // (browser language != farmer's preferred language for this
app) // On language change from header dropdown: const handleLanguageChange = async (newLang:
string) => { // 1. Update UI immediately (optimistic) setLocale(newLang); // 2. Save to backend
await api.patch('/users/me', { language_code: newLang }); // 3. Set Accept-Language header for all
future requests apiClient.defaults.headers['Accept-Language'] = newLang; };
```

9.3 String Keys Convention

All i18n string keys use dot notation with a screen or domain prefix:

```
// Key naming convention "advisory.order_now" // Advisory screen, order button
"advisory.conditional_blank" // Conditional blank warm message "orders.awaiting_approval" //
Orders tab status "qr.verified_badge" // QR verified badge text "qr.mismatch_message" // QR
mismatch nudge "common.retry" // Shared: retry button "common.cancel" // Shared: cancel button
"navigation.home" // Navigation: home "empty_state.no_advisories" // Empty state text
```

9.4 Translation Pending Indicator

When the API returns content with a translation_pending flag (PENDING status translation), the frontend renders a subtle indicator:

```
// TranslatableText component pattern: // - Render text in farmer language // - If isPending: show
subtle note below // - Note text from i18n: t("common.translation_pending") // - Style: small,
grey, italic -- non-alarming
```

9.5 Font Loading Strategy

Indian scripts require specific fonts. Load them conditionally to avoid performance impact for users who do not need them:

Script	Font	Languages	Loading Strategy
Devanagari	Noto Sans Devanagari	Hindi, Marathi	Lazy load when lang=hi or lang=mr
Tamil	Noto Sans Tamil	Tamil	Lazy load when lang=ta
Telugu	Noto Sans Telugu	Telugu	Lazy load when lang=te
Kannada	Noto Sans Kannada	Kannada	Lazy load when lang=kn
Malayalam	Noto Sans Malayalam	Malayalam	Lazy load when lang=ml
Gujarati	Noto Sans Gujarati	Gujarati	Lazy load when lang=gu
Gurmukhi	Noto Sans Gurmukhi	Punjabi	Lazy load when lang=pa
Odia	Noto Sans Oriya	Odia	Lazy load when lang=or
Bengali	Noto Sans Bengali	Bengali, Assamese	Lazy load when lang=bn or lang=as
Urdu (Nastaliq)	Noto Nastaliq Urdu	Urdu	Lazy load when lang=ur. Also sets dir=rtl.
Latin	System font / Inter	English	Always loaded (default)

■ **Note:** Use the Google Fonts preconnect hint and font-display: swap for all lazy-loaded fonts. Test font loading on 4G and 3G connections common in rural India.

10. Notification Language Handling

10.1 FCM Push Notifications

Every FCM push notification must be sent in the recipient's language. The notification dispatch service looks up the recipient's language_code before constructing the notification payload.

```
# Notification dispatch service
def send_push_notification(user_id: str, template_key: str,
    variables: dict):
    user = db.get_user(user_id)
    lang = user.language_code # e.g. 'kn'
    # Load translated template
    title = get_notification_text(template_key + '.title', lang, variables)
    body = get_notification_text(template_key + '.body', lang, variables)
    # Notification text templates are stored in the notification_templates # table
    # (one row per template_key per language)
    send_fcm(user.fcm_token, title=title, body=body)
```

10.2 Notification Templates Table

Unlike system UI strings (which live in JSON files), notification templates are stored in the database to allow the backend to look them up at dispatch time without importing frontend assets:

```
CREATE TABLE notification_templates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    template_key VARCHAR(100) NOT NULL,
    -- e.g. 'FCM-F-01'
    language_code VARCHAR(10) NOT NULL,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    UNIQUE (template_key, language_code) );
```

This table is seeded with English templates for all 22 FCM notifications (from the Notification Templates document). Additional languages are added by the Neytiri content team as translations are approved.

10.3 Email Language

All emails are sent in English only. No translation needed for emails. See Notification Templates document §2.

11. Localisation Implementation Checklist

Before enabling any non-English language in staging

- All i18n string keys translated in the target language file. No placeholder English values remaining.
- All Cosh reference data for the target language available in cosh_reference_cache (verify after Cosh sync).
- IndicTrans2 or Google Translate API tested for that language with RootsTalk-specific agricultural content.
- Notification templates table seeded for the new language.
- Font for the target script lazy-loaded and tested on a real device.
- All date and number formatters tested with the new locale.
- If Urdu: RTL layout verified on Android and iOS real devices.
- Advisory screen tested with long text in the target language (some scripts are wider than Latin — check for UI overflow).
- Language selector tested: can a farmer who does not read English find and select their language?

Per-developer rules during development

- Never hard-code display strings in React components. Always use t('key').

- When adding a new UI string: add to en.json first, then add the key (with English placeholder) to all other language files. Mark with TODO: translate.
- Never add a language file key without adding it to en.json first.
- When adding a new notification: add the English template to the notification_templates table seed data immediately.
- Test every new screen in at least two non-English languages (suggest Kannada and Hindi as default test languages) before marking the screen as complete.
- RTL: test every new screen in Urdu (dir=rtl) before marking complete.